

SCS 1309

Database Management Systems

Lecture 3 – Structured Query Language (SQL)

MVPT Lakshika,
Lecturer (Probationary),
Department of Information Systems Engineering, UCSC.
Contact: tlv@ucsc.cmb.ac.lk

Outline of this session...

1. SQL Aggregate Functions
2. Grouping
3. Subqueries
4. ANY and ALL
5. EXISTS and NOT EXISTS
6. Multi-Table Queries
7. Computing a join
8. Combining Result Tables

1. SQL Aggregate Functions

Introduction

- Apart from retrieving rows and columns from the database, we often want to perform some form of **summation or aggregation** of data.
- The ISO standard defines **five aggregate functions**:
 1. COUNT – returns the number of values in a specified column
 2. SUM – returns the sum of the values in a specified column
 3. AVG – returns the average of the values in a specified column
 4. MIN – returns the smallest value in a specified column
 5. MAX – returns the largest value in a specified column
- COUNT, MIN, and MAX apply to both numeric and non-numeric fields, but SUM and AVG may be used on numeric fields only.

Introduction

Each aggregate function,

- Operate on a **single column** of a table and **return a single value**.
- **Eliminates nulls first** and operates only on the remaining non-null values.
- Can be used only in the **SELECT list** and in the **HAVING clause**. It is incorrect to use it elsewhere.
- If the SELECT list includes an aggregate function, and no GROUP BY clause is being used to group data together, then no item in the SELECT list can include any reference to a column unless that column is the argument to an aggregate function.

Example:

```
SELECT staffNo, COUNT(salary)  
FROM Staff;
```

This query is illegal.

Use of COUNT()

- **COUNT(*)** is a special use of COUNT, which counts all the rows of a table, regardless of whether nulls or duplicate values occur.

Example 1:

How many properties cost more than 350 per month to rent?

```
SELECT COUNT(*) AS myCount
FROM PropertyForRent
WHERE rent > 350;
```

PropertyForRent

propertyNo	street	city	postcode	type	rooms	rent	ownerNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93

Result:

myCount
5

Use of COUNT(DISTINCT)

- Use the keyword **DISTINCT** before the column name in the function to eliminate duplicates before the function is applied.
- The ISO standard allows the keyword **ALL** to be specified if you do not want to eliminate duplicates, although **ALL** is assumed if nothing is specified.
- **DISTINCT** does not affect the **MIN** and **MAX** functions, but affects the result of **SUM** or **AVG**.
- In addition, **DISTINCT** can be specified *only once in a query*.

Example 2:

How many different properties were viewed in May 2004?

Viewing

clientNo	propertyNo	viewDate	comment
CR56	PA14	24-May-04	too small
CR76	PG4	20-Apr-04	too remote
CR56	PG4	26-May-04	no dining room
CR62	PA14	14-May-04	
CR56	PG36	28-Apr-04	

SELECT COUNT(DISTINCT propertyNo) AS myCount
FROM Viewing
WHERE viewDate BETWEEN '1-May-04' AND '31-May-04';

Result:

myCount
2

Use of COUNT and SUM

Example 3:

Find the total number of Managers and the sum of their salaries.

```
SELECT COUNT(staffNo) AS myCount, SUM(salary) AS mySum  
FROM Staff  
WHERE position = 'Manager';
```

Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

Result:

myCount	mySum
2	54000.00

Use of MIN, MAX, AVG

Example 4:

Find the minimum, maximum, and average staff salary.

```
SELECT MIN(salary) AS myMin, MAX(salary) AS myMax, AVG(salary) AS myAvg  
FROM Staff;
```

- Consider all staff and therefore do not require a WHERE clause.

Result:

myMin	myMax	myAvg
9000.00	30000.00	17000.00

Activity 1

How many female staff members are working in the branch B003?

```
SELECT COUNT(*) AS CountFemale  
FROM staff  
WHERE sex = 'F' AND branchNo = 'B003';
```

How many flats are located in cities starting with the letter **G**?

```
SELECT COUNT(type) as countFlats  
FROM PropertyForRent  
WHERE type = 'Flat' AND city LIKE 'G%';
```

What is the maximum and minimum rental price of a flat located in Glasgow?

```
SELECT MAX(rent) AS maxRent, MIN(rent) AS minRent  
FROM PropertyForRent  
WHERE type = 'Flat' AND city = 'Glasgow';
```

2. Grouping

Introduction

- A query that includes the **GROUP BY** clause is called a **grouped query**, because it groups the data from the SELECT table(s) and produces **a single summary row for each group**.
- The columns named in the **GROUP BY** clause are called the **grouping columns**.
- When GROUP BY is used, each item in the SELECT list must be **single-valued per group**.
- Further, the SELECT clause may contain only:
 - Column names
 - Aggregate functions
 - Constants (String Constants, Numeric Constants, Null Constants)
 - An expression involving combinations of the above.

Introduction

- All column names in the SELECT list must appear in the GROUP BY clause unless the name is used only in an aggregate function.

Example: **SELECT** branchNo, **COUNT**(staffNo) **AS** totalStaff, 'Staff Report' **AS** ReportType

FROM Staff

GROUP BY branchNo;

- There may be column names in the GROUP BY clause that do not appear in the SELECT list.
- When the WHERE clause is used with GROUP BY, the WHERE clause is applied first, then groups are formed from the remaining rows that satisfy the search condition.
- The ISO standard considers two nulls to be equal for purposes of the GROUP BY clause.
- If two rows have nulls in the same grouping columns and identical values in all the non-null grouping columns, they are combined into the same group.

Use of GROUP BY

Example 5:

Find the number of staff working in each branch and the sum of their salaries.

```
SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum
FROM Staff
GROUP BY branchNo
ORDER BY branchNo;
```

Staff

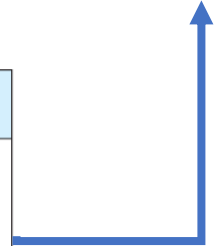
staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005



branchNo	staffNo	salary
B003	SG37	12000.00
B003	SG14	18000.00
B003	SG5	24000.00
B005	SL21	30000.00
B005	SL41	9000.00
B007	SA9	9000.00



COUNT(staffNo)	SUM(salary)
3	54000.00
2	39000.00
1	9000.00



Result:

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00
B007	1	9000.00

Activity 2

Find the number of properties for rent in each city, ordered alphabetically by city.

```
SELECT city, COUNT(*) AS noOfProperties  
FROM PropertyForRent  
GROUP BY city  
ORDER BY city ASC;
```

Find the minimum and maximum salaries for both female and male staff members.

```
SELECT sex, MIN(salary) as minSalry, MAX(salary) AS maxSalary  
FROM staff  
GROUP BY sex;
```

Restricting groupings

- The **HAVING** clause is designed for use with the GROUP BY clause to **restrict the groups that appear in the final result table**.
- Although similar in syntax, HAVING and WHERE serve different purposes.
 - The WHERE clause filters **individual rows** going into the final result table.
 - HAVING filter **groups** going into the final result table.
- The ISO standard requires that column names used in the HAVING clause must also appear in the **GROUP BY list** or be **contained within an aggregate function**.
- The search condition in the HAVING clause always includes at least one aggregate function, otherwise, the search condition could be moved to the WHERE clause and applied to individual rows.
- The HAVING clause is not a necessary part of SQL – **any query expressed using a HAVING clause can always be rewritten without the HAVING clause**.

Use of HAVING

Example 6:

For each branch office with more than one member of staff, find the number of staff working in each branch and the sum of their salaries.

```
SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum
```

```
FROM Staff
```

```
GROUP BY branchNo
```

```
HAVING COUNT(staffNo) > 1
```

```
ORDER BY branchNo;
```

- This is similar to example 5, with the additional restriction that we want to consider only those groups (branches) with more than one member of staff.

Result (Example 5):

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00
B007	1	9000.00

Result (Example 6):

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00

Activity 3

Find positions within the company that have an average salary greater than 15,000.

```
SELECT position, AVG(salary) AS avgSalary  
FROM Staff  
GROUP BY position  
HAVING AVG(salary) > 15000;
```

Find the cities where the average rent exceeds 500 for House-type properties.

```
SELECT city, AVG(rent) as avg_rent  
FROM PropertyForRent  
WHERE type = 'House'  
GROUP BY city  
HAVING AVG(rent) > 500;
```

3. Subqueries

Introduction

- Subqueries include a complete SELECT statement embedded within another SELECT statement.
- The results of this **inner** SELECT statement (or **subselect**) are used in the **outer** statement to help determine the contents of the final result.
 - A subselect can be used immediately following a relational operator (=, <, >, <=, >=, < >) in the WHERE and HAVING clauses of an outer SELECT statement, where it is called a **subquery** or **nested query**.
 - Subselects may also appear in INSERT, UPDATE, and DELETE statements.
- There are **three types** of subquery: **Scalar subquery**, **Row subquery**, **Table subquery**

Scalar subquery

- Returns a **single value** (single column and a single row).
- A scalar subquery can be used whenever a single value is needed.

Example 7:

List the staff who work in the branch at '163 Main St'.

```
SELECT staffNo, fName, lName, position
```

```
FROM Staff
```

```
WHERE branchNo = (SELECT branchNo  
                  FROM Branch  
                  WHERE street = '163 Main St');
```

subquery

Result:

staffNo	fName	lName	position
SG37	Ann	Beech	Assistant
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

Scalar subquery

Example 8:

List all staff whose salary is greater than the average salary, and show by how much their salary is greater than the average.

```
SELECT staffNo, fName, lName, position, salary – (SELECT AVG(salary) FROM Staff) AS  
salDiff  
FROM Staff  
WHERE salary > (SELECT AVG(salary) FROM Staff);
```

- You cannot write 'WHERE salary > AVG(salary)' because aggregate functions cannot be used in the WHERE clause.
- Instead, use a subquery to find the average salary, and then use the outer SELECT statement to find those staff with a salary greater than this average.

Scalar subquery

Example 8 (Cont...) :

- The subquery returns the average salary as 17,000. Then the outer query is reduced to:

SELECT staffNo, fName, lName, position, salary – 17000 **AS** salDiff

FROM Staff

WHERE salary > 17000;

Result:

staffNo	fName	lName	position	salDiff
SL21	John	White	Manager	13000.00
SG14	David	Ford	Supervisor	1000.00
SG5	Susan	Brand	Manager	7000.00

Row subquery

- Returns multiple columns, but again only a single row.
- A row subquery can be used whenever a row value constructor is needed, typically in predicates.

Example 9:

Find staff members who have the same position and salary as *David Ford (SG14)*.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE (position, salary) = (SELECT position, salary
                             FROM Staff
                             WHERE staffNo = 'SG14') AND staffNo <> 'SG14';
```

Table subquery

- Returns one or more columns and multiple rows.
- A table subquery can be used whenever a table is needed.
- In this case, you cannot use the equality condition (=) in the outermost query. Instead, use the IN keyword.

Example 10: List the full name and address of the clients who have viewed properties.

```
SELECT clientNo, CONCAT(fName, ' ', lName) AS name, telNo
FROM Client
WHERE clientNo IN ( SELECT clientNo
                    FROM Viewing);
```

Nested subqueries: use of IN

Example 11:

List the properties that are handled by staff who work in the branch at '163 Main St'.

Consider the schema for **PropertyForRent**:

PropertyForRent (propertyNo, street, city, postcode, type, rooms, rent, ownerNo, staffNo)

```
SELECT propertyNo, street, city, postcode, type, rooms, rent
FROM PropertyForRent
WHERE staffNo IN (SELECT staffNo
                  FROM Staff
                  WHERE branchNo = (SELECT branchNo
                                     FROM Branch
                                     WHERE street = '163 Main St'));
```

4. ANY and ALL

Introduction

- The words **ANY** and **ALL** may be used with subqueries that produce a **single column of numbers**.
 - If the subquery is preceded by the keyword **ALL**, the condition will only be true if it is satisfied by all values produced by the subquery.
 - If the subquery is preceded by the keyword **ANY**, the condition will be true if it is satisfied by any (one or more) values produced by the subquery.
 - If the subquery is empty, the **ALL** condition returns true, the **ANY** condition returns false.
- The ISO standard also allows the qualifier **SOME** to be used in place of **ANY**.

Use of ANY/SOME

Example 12:

Find all staff whose salary is larger than the salary of at least one member of staff at branch B003.

Approach 1

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > (SELECT MIN(salary)
                FROM Staff
                WHERE branchNo = 'B003');
```

Approach 2

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > SOME (SELECT salary
                       FROM Staff
                       WHERE branchNo = 'B003');
```

Result:

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00

Use of ALL

Example 13:

Find all staff whose salary is larger than the salary of every member of staff at branch B003.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > ALL (SELECT salary
                     FROM Staff
                     WHERE branchNo = 'B003');
```

Result:

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00

- Again, you could use a subquery to find the maximum salary of staff at branch B003 and then use an outer query to find all staff whose salary is greater than this number.

5. EXISTS and NOT EXISTS

Introduction

- The keywords **EXISTS** and **NOT EXISTS** are designed for use only with subqueries.
- They produce a simple true/false result.
 - **EXISTS** is true if and only if there exists at least one row in the result table returned by the subquery; it is false if the subquery returns an empty result table.
 - **NOT EXISTS** is the opposite of EXISTS.
- Since EXISTS and NOT EXISTS check only for the existence or non-existence of rows in the subquery result table, the subquery can contain any number of columns.
- For simplicity, it is common for subqueries following one of these keywords to be of the form:

(SELECT * FROM . . .)

Query using EXISTS

Example 14

Find all staff who work in a London branch office (Find all staff such that there exists a Branch row containing his/her branch number, branchNo, and the branch city equal to London)

SELECT staffNo, fName, lName, position

FROM Staff s

WHERE EXISTS (SELECT *

FROM Branch b

WHERE s.branchNo = b.branchNo **AND** city = 'London');

Result:

staffNo	fName	lName	position
SL21	John	White	Manager
SL41	Julie	Lee	Assistant

- The test for inclusion is the existence of such a row. If it exists, the subquery evaluates to true.

Query using EXISTS

Example 14 (Cont...)

- The `s.branchNo = b.branchNo` in the search condition is necessary to ensure that you consider the correct branch row for each member of staff.
 - If you omitted the search condition, you would get all staff rows listed out.

Because the subquery (`SELECT * FROM Branch WHERE city = 'London'`) would always be true, and the query would be reduced to:

```
SELECT staffNo, fName, lName, position  
FROM Staff  
WHERE true;
```



```
SELECT staffNo, fName, lName, position  
FROM Staff;
```

- The same query can be written using the join construct:

```
SELECT staffNo, fName, lName, position  
FROM Staff s, Branch b  
WHERE s.branchNo = b.branchNo AND city = 'London';
```

Query using NOT EXISTS

- **NOT EXISTS** operator is used to return rows from one table where no matching record is found in a subquery.
- Unlike NOT IN, which compares specific values, NOT EXISTS checks for the absence of any rows that meet the criteria.

Example 15

Find properties that have **never been viewed** by any client.

```
SELECT p.propertyNo, p.street, p.city
FROM PropertyForRent p
WHERE NOT EXISTS (SELECT 1
                    FROM Viewing v
                    WHERE v.propertyNo = p.propertyNo);
```

6. Multi-Table Queries

Introduction

- To combine columns from several tables into a result table, you need to use a [join](#) operation.
- The [SQL join](#) operation combines information from two tables by forming pairs of related rows from the two tables.
 - Matching columns in each of the two tables have the same value.
- When to join tables?
 - If you need to obtain information from more than one table, the choice is between using a subquery and using a join.
 - If the final result table is to contain columns from different tables, then you must use a join.

Introduction

- To perform a join, **include more than one table name in the FROM clause**, using a comma as a separator, and **include a WHERE clause to specify the join column(s)**.
- It is also possible to use an **alias** for a table named in the FROM clause.
- An alias can be used to qualify a column name whenever there is ambiguity regarding the source of the column name.
- It can also be used as a shorthand notation for the table name.
- If an alias is provided, it can be used anywhere in place of the table name.

Example:

```
SELECT c.clientNo, c.fName, c.lName, v.propertyNo, v.comment  
FROM Client c, Viewing v  
WHERE c.clientNo = v.clientNo;
```

Simple join

Client

clientNo	fName	lName	address	telNo	prefType	maxRent
CR76	John	Kay	56 High St, London SW1 4EH	0207-774-5632	Flat	425
CR56	Aline	Stewart	64 Fern Dr, Glasgow G42 0BL	0141-848-1825	Flat	350
CR74	Mike	Ritchie	18 Tain St, PA1G 1YQ	01475-392178	House	750
CR62	Mary	Tregear	5 Tarbot Rd, Aberdeen AB9 3ST	01224-196720	Flat	600

Example 16: List the names of all clients who have viewed a property, along with any comments supplied.

- Results should display the details from both the **Client** table and the **Viewing** table.
- **clientNo**, in the SELECT list, could come from either table, and you have to indicate which one.
- The columns in the search condition (**c.clientNo = v.clientNo**) are the matching columns for the two tables.

SELECT **c**.clientNo, fName, lName, propertyNo, comment

FROM Client **c**, Viewing **v**

WHERE **c**.clientNo = **v**.clientNo;

Viewing

clientNo	propertyNo	viewDate	comment
CR56	PA14	24-May-04	too small
CR76	PG4	20-Apr-04	too remote
CR56	PG4	26-May-04	
CR62	PA14	14-May-04	no dining room
CR56	PG36	28-Apr-04	

Result:

clientNo	fName	lName	propertyNo	comment
CR56	Aline	Stewart	PG36	
CR56	Aline	Stewart	PA14	too small
CR56	Aline	Stewart	PG4	
CR62	Mary	Tregear	PA14	no dining room
CR76	John	Kay	PG4	too remote

Simple join (Cont...)

- The most common multi-table queries involve two tables that have a **one-to-many (1:*)** (or a parent/child) relationship.
- The pairs of rows that generate the query results are parent/child row combinations.
- The table containing the primary key is the parent table, and the table containing the foreign key is the child table.
- Considering example 16;
 - Each viewing (child) has an associated client (parent), and each client (parent) can have many associated viewings (children).
 - The search condition compares the primary key (in the Client table, c.clientNo)and the foreign key (in the Viewing table, v.clientNo).

Simple join (Cont...)

- The SQL standard provides the following alternative ways to specify this join:

SELECT **c**.clientNo, fName, lName, propertyNo, comment

FROM Client **c**, Viewing **v**

WHERE **c**.clientNo = **v**.clientNo;

FROM Client c **JOIN** Viewing v **ON** c.clientNo = v.clientNo

FROM Client **JOIN** Viewing **USING** clientNo

FROM Client **NATURAL JOIN** Viewing

- In each case, the FROM clause replaces the original FROM and WHERE clauses.
- However, the first alternative produces a table with two identical clientNo columns; the remaining two produce a table with a single clientNo column.

Sorting a join

Example 17:

For each branch office, list the staff numbers and names of staff who manage properties and the properties that they manage.

```
SELECT s.branchNo, s.staffNo, fName, lName, propertyNo
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
ORDER BY s.branchNo, s.staffNo, propertyNo;
```

Result:

branchNo	staffNo	fName	lName	propertyNo
B003	SG14	David	Ford	PG16
B003	SG37	Ann	Beech	PG21
B003	SG37	Ann	Beech	PG36
B005	SL41	Julie	Lee	PL94
B007	SA9	Mary	Howe	PA14

- To make the results more readable, this query has ordered the output using the **branch number** as the **major sort key** and the **staff number** and **property number** as the **minor keys**.

Three-table join

Example 18:

For each branch, list the staff number and the name of staff who manage properties, including the city in which the branch is located and the properties that the staff manage.

```
SELECT b.branchNo, b.city, s.staffNo, fName, lName,
propertyNo
```

```
FROM Branch b, Staff s, PropertyForRent p
```

```
WHERE b.branchNo = s.branchNo AND s.staffNo =
p.staffNo
```

```
ORDER BY b.branchNo, s.staffNo, propertyNo;
```

Result:

Branch

branchNo	street	city	postCode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

PropertyForRent

propertyNo	street	city	postcode	rent
PA14	16 Holhead	Aberdeen	AB7 5SU	650
PL94	6 Argyll St	London	NW2	400
PG21	18 Dale Rd	Glasgow	G12	600

Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

branchNo	city	staffNo	fName	lName	propertyNo
B003	Glasgow	SG14	David	Ford	PG16
B003	Glasgow	SG37	Ann	Beech	PG21
B003	Glasgow	SG37	Ann	Beech	PG36
B005	London	SL41	Julie	Lee	PL94
B007	Aberdeen	SA9	Mary	Howe	PA14

Three-table join

Example 18 (Cont...):

- The **Branch** and **Staff** details are joined using the condition (b.branchNo = s.branchNo), to link each branch to the staff who work there.
- The **Staff** and **PropertyForRent** details are joined using the condition (s.staffNo = p.staffNo), to link staff to the properties they manage.
- The SQL standard provides alternative formulations for the FROM and WHERE clauses, for example:

```
FROM (Branch b JOIN Staff s USING branchNo) AS bs JOIN PropertyForRent p  
USING staffNo
```

Multiple grouping columns

Example 19:

List each branch number and staff number along with the number of properties handled by each staff member.

```
SELECT s.branchNo, s.staffNo, COUNT(*) AS myCount
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.branchNo, s.staffNo
ORDER BY s.branchNo, s.staffNo;
```

Result:

branchNo	staffNo	myCount
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1

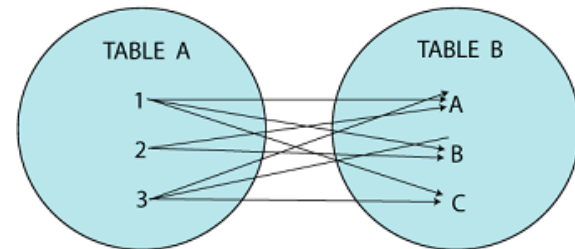
- First, find out which staff actually manage properties by joining the Staff and PropertyForRent tables on the staffNo column.
- Next, form groups consisting of the branch number and staff number, using the GROUP BY clause.
- Finally, sort the output using the ORDER BY clause.

7. Computing a join

Introduction

- A join is a subset of a more general combination of two tables known as the **Cartesian product**.
- The Cartesian product of two tables is,
 - Another table consisting of **all possible pairs of rows from the two tables**.
 - The columns of the product table are all the columns of the first table, followed by all the columns of the second table.
- If you specify a two-table query without a WHERE clause, SQL produces the Cartesian product of the two tables as the query result.
- ISO standard provides a special form of the SELECT statement for the Cartesian product:

```
SELECT  [DISTINCT | ALL] { * | columnList }  
FROM    TableName1 CROSS JOIN TableName2
```



Introduction (Cont...)

- Consider the Cartesian product of tables: Client and Viewing

SELECT *

FROM Client **CROSS JOIN** Viewing;

Client

clientNo	fName	lName	address	telNo	prefType	maxRent
CR76	John	Kay	56 High St, London SW1 4EH	0207-774-5632	Flat	425
CR56	Aline	Stewart	64 Fern Dr, Glasgow G42 0BL	0141-848-1825	Flat	350
CR74	Mike	Ritchie	18 Tain St, PA1G 1YQ	01475-392178	House	750
CR62	Mary	Tregear	5 Tarbot Rd, Aberdeen AB9 3ST	01224-196720	Flat	600

Viewing

clientNo	propertyNo	viewDate	comment
CR56	PA14	24-May-04	too small
CR76	PG4	20-Apr-04	too remote
CR56	PG4	26-May-04	
CR62	PA14	14-May-04	no dining room
CR56	PG36	28-Apr-04	

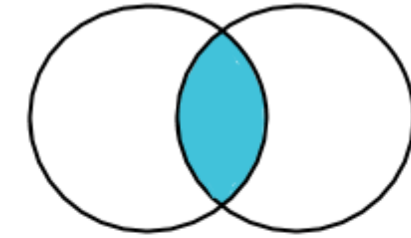
- The Cartesian product of these two tables would contain 20 rows (4 clients * 5 viewings = 20 rows).

Introduction (Cont...)

The procedure for generating the results of a SELECT with a join is as follows:

1. Form the Cartesian product of the tables named in the FROM clause.
2. If there is a WHERE clause, apply the search condition to each row of the product table, retaining those rows that satisfy the condition.
3. For each remaining row, determine the value of each item in the SELECT list to produce a single row in the result table.
4. If SELECT DISTINCT has been specified, eliminate any duplicate rows from the result table.
5. If there is an ORDER BY clause, sort the result table as required.

Inner Join



Inner Join

- Consider the following two simplified **Branch** and **PropertyForRent** tables.

branchNo	bCity
B003	Glasgow
B004	Bristol
B002	London

propertyNo	pCity
PA14	Aberdeen
PL94	London
PG4	Glasgow

Inner join

branchNo	bCity	propertyNo	pCity
B003	Glasgow	PG4	Glasgow
B002	London	PL94	London

- The **Inner join** of these two tables:

SELECT b.*, p.*

FROM Branch1 b, PropertyForRent1 p

WHERE b.bCity = p.pCity;

- The result table has two rows where the cities are the same.
- There is no row corresponding to the branch office in Bristol and Aberdeen.

Outer joins

- Consider the previous inner join example. If you want to include the unmatched rows in the result table?
- To support that, the ISO standard provides another set of join operators called outer joins.
- When to use an Outer join?
 - If you need to include the unmatched rows in the result table (Outer join retains rows that do not satisfy the join condition)
- There are three types of outer joins: **Left**, **Right**, and **Full Outer joins**.

Left Outer join

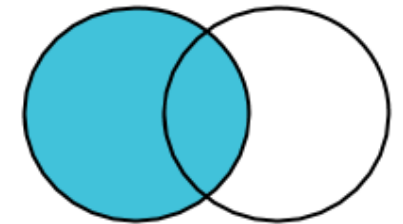
Example 20

List all branch offices and any properties that are in the same city.

- The Left Outer join of these two tables:

SELECT b.*, p.*

FROM Branch1 b LEFT JOIN PropertyForRent1 p ON b.bCity = p.pCity;



Left Join

Branch1		PropertyForRent1		Result:			
branchNo	bCity	propertyNo	pCity	branchNo	bCity	propertyNo	pCity
B003	Glasgow	PA14	Aberdeen	B003	Glasgow	PG4	Glasgow
B004	Bristol	PL94	London	B004	Bristol	NULL	NULL
B002	London	PG4	Glasgow	B002	London	PL94	London

Left outer join

- Results include not only those rows that have the same city, but also those rows of the first (left) table that are unmatched with rows from the second (right) table.
- The columns from the second table are filled with NULLs.

Right Outer join

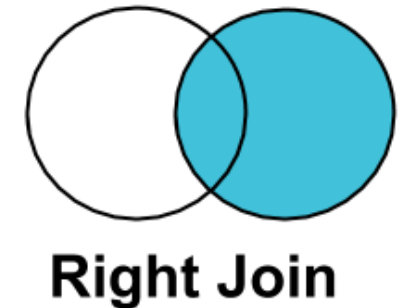
Example 21

List all properties and any branch offices that are in the same city.

- The Right Outer join of these two tables:

```
SELECT b.*, p.*
```

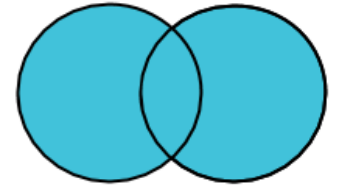
```
FROM Branch1 b RIGHT JOIN PropertyForRent1 p ON b.bCity = p.pCity;
```



Branch1		PropertyForRent1		Result:			
branchNo	bCity	propertyNo	pCity	branchNo	bCity	propertyNo	pCity
B003	Glasgow	PA14	Aberdeen	NULL	NULL	PA14	Aberdeen
B004	Bristol	PL94	London	B003	Glasgow	PG4	Glasgow
B002	London	PG4	Glasgow	B002	London	PL94	London

- Results include not only those rows that have the same city, but also those rows of the second (right) table that are unmatched with rows from the first (left) table.
- The columns from the first table are filled with NULLs.

Full Outer join



**Full Outer
Join**

Example 22

List the branch offices and properties that are in the same city, along with any unmatched branches or properties.

- The Full Outer join of these two tables:

```
SELECT b.*, p.*
```

```
FROM Branch1 b FULL JOIN PropertyForRent1 p ON b.bCity = p.pCity;
```

Branch1		PropertyForRent1					
branchNo	bCity	propertyNo	pCity				
B003	Glasgow	PA14	Aberdeen				
B004	Bristol	PL94	London				
B002	London	PG4	Glasgow				

Full outer join →

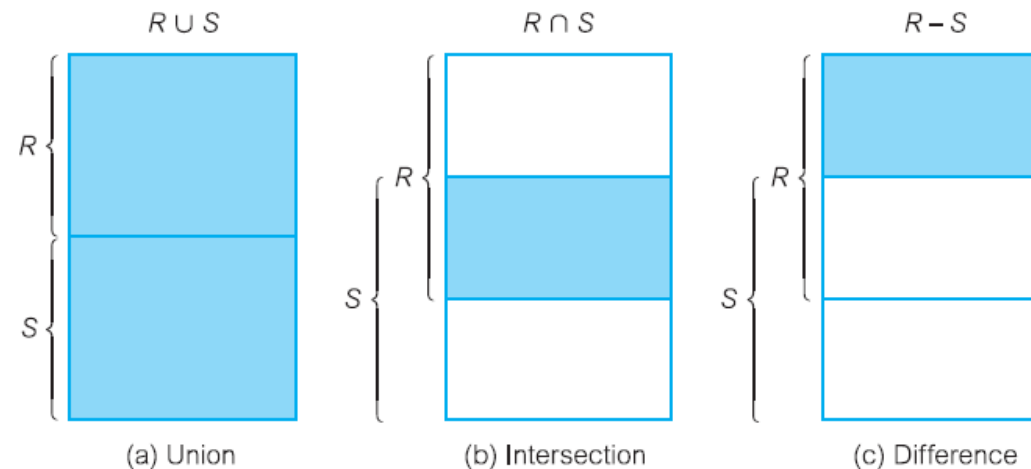
Result:			
branchNo	bCity	propertyNo	pCity
NULL	NULL	PA14	Aberdeen
B003	Glasgow	PG4	Glasgow
B004	Bristol	NULL	NULL
B002	London	PL94	London

- Results include not only those rows that have the same city, but also those rows that are unmatched in both tables.
- The unmatched columns are filled with NULLs.

8. Combining Result Tables

Introduction

- In SQL, you can use the normal set operations of **Union**, **Intersection**, and **Difference** to combine the results of **two or more queries** into a single result table:
 - The **Union** of two tables, A and B, is a table containing all rows that are in either the first table A or the second table B or both.
 - The **Intersection** of two tables, A and B, is a table containing all rows that are common to both tables A and B.
 - The **Difference** of two tables, A and B, is a table containing all rows that are in table A but are not in table B.



Introduction

- There are restrictions on the tables that can be combined using the set operations.
 - The two tables have to be **union-compatible** (should have the same structure)
 - Two tables must contain the same number of columns
 - The corresponding columns have the same data types and lengths.
- It is the user's responsibility to ensure that data values in corresponding columns come from the same *domain*.

Example: It is not sensible to combine a column containing the age of staff with the number of rooms in a property, even though both columns may have the same data type.

Introduction

- The three set operators in the ISO standard are called **UNION**, **INTERSECT**, and **EXCEPT**.
- The format of the set operator clause in each case is:

operator [**ALL**] [**CORRESPONDING** [**BY** {column1 [, . . .]}]]

- If **CORRESPONDING BY** is specified, then the set operation is performed on the named column(s).
- If **CORRESPONDING** is specified but not the **BY** clause, the set operation is performed on the columns that are common to both tables.
- If **ALL** is specified, the result can include duplicate rows.
- Some dialects of SQL do not support **INTERSECT** and **EXCEPT**; others use **MINUS** in place of **EXCEPT**.

Use of UNION

Example 23:

Construct a list of all cities where there is either a branch office or a property.

```
SELECT city
FROM Branch
WHERE city IS NOT NULL
```

UNION

```
SELECT city
FROM PropertyForRent
WHERE city IS NOT NULL;
```

or

```
(SELECT *
FROM Branch
WHERE city IS NOT NULL)
```

UNION CORRESPONDING BY city

```
(SELECT *
FROM PropertyForRent
WHERE city IS NOT NULL);
```

Result:

city
London
Glasgow
Aberdeen
Bristol

- This query is executed by producing a result table from the first query and a result table from the second query, and then merging both tables into a single result table consisting of all the rows from both result tables **with the duplicate rows removed**.

Use of UNION

Example 24:

Construct a list of all cities where there is either a branch office or a property.
Retains all duplicate entries in the final result list.

```
SELECT city
FROM Branch
WHERE city IS NOT NULL
UNION ALL
SELECT city
FROM PropertyForRent
WHERE city IS NOT NULL;
```

Result:

city
London
Glasgow
Bristol
London
Aberdeen
Aberdeen
Glasgow
Glasgow
Glasgow
Glasgow
London

Use of INTERSECT

Example 25:

Construct a list of all cities where there is both a branch office and a property.

```
SELECT city  
FROM Branch  
INTERSECT
```

or

```
SELECT city  
FROM PropertyForRent;
```

```
SELECT *  
FROM Branch  
INTERSECT CORRESPONDING BY city  
SELECT *  
FROM PropertyForRent;
```

Result:

city
Aberdeen
Glasgow
London

- This query is executed by producing a result table from the first query and a result table from the second query, and then creating a single result table consisting of those rows that **are common to both result tables**.

Use of INTERSECT

How to achieve the same result in Example 25 without **INTERSECT** ?

Using a **WHERE** clause

```
SELECT DISTINCT b.city  
FROM Branch b, PropertyForRent p  
WHERE b.city = p.city;
```

Using a **JOIN**

```
SELECT DISTINCT b.city  
FROM Branch b  
INNER JOIN PropertyForRent p ON b.city = p.city;
```

Using the **IN** operator

```
SELECT DISTINCT city  
FROM Branch  
WHERE city IN (SELECT city  
                FROM PropertyForRent);
```

Using the **EXISTS** operator

```
SELECT DISTINCT city  
FROM Branch b  
WHERE EXISTS (SELECT *  
              FROM PropertyForRent p  
              WHERE b.city = p.city);
```

The ability to write a query in several equivalent forms illustrates one of the disadvantages of the SQL language.

Use of EXCEPT

Example 26:

Construct a list of all cities where there is a branch office but no properties.

```
SELECT city  
FROM Branch
```

EXCEPT

```
SELECT city  
FROM PropertyForRent;
```

or

```
(SELECT *  
FROM Branch)
```

EXCEPT CORRESPONDING BY city

```
(SELECT *  
FROM PropertyForRent);
```

Result:

city
Bristol

- This query is executed by producing a result table from the first query and a result table from the second query, and then creating a single result table consisting of those rows that **appear in the first result table but not in the second one.**

Use of EXCEPT

How to achieve the same result in Example 26 without **EXCEPT** ?

Using the **NOT IN** operator

```
SELECT DISTINCT city  
FROM Branch  
WHERE city NOT IN (SELECT DISTINCT city  
                     FROM PropertyForRent  
                     WHERE city IS NOT NULL);
```

Using a **NOT EXISTS**

```
SELECT DISTINCT city  
FROM Branch b  
WHERE NOT EXISTS (SELECT *  
                  FROM PropertyForRent p  
                  WHERE b.city = p.city);
```

Using a **JOIN**

```
SELECT DISTINCT b.city  
FROM Branch b  
LEFT JOIN PropertyForRent p ON b.city = p.city  
WHERE p.city IS NULL;
```

Any questions?